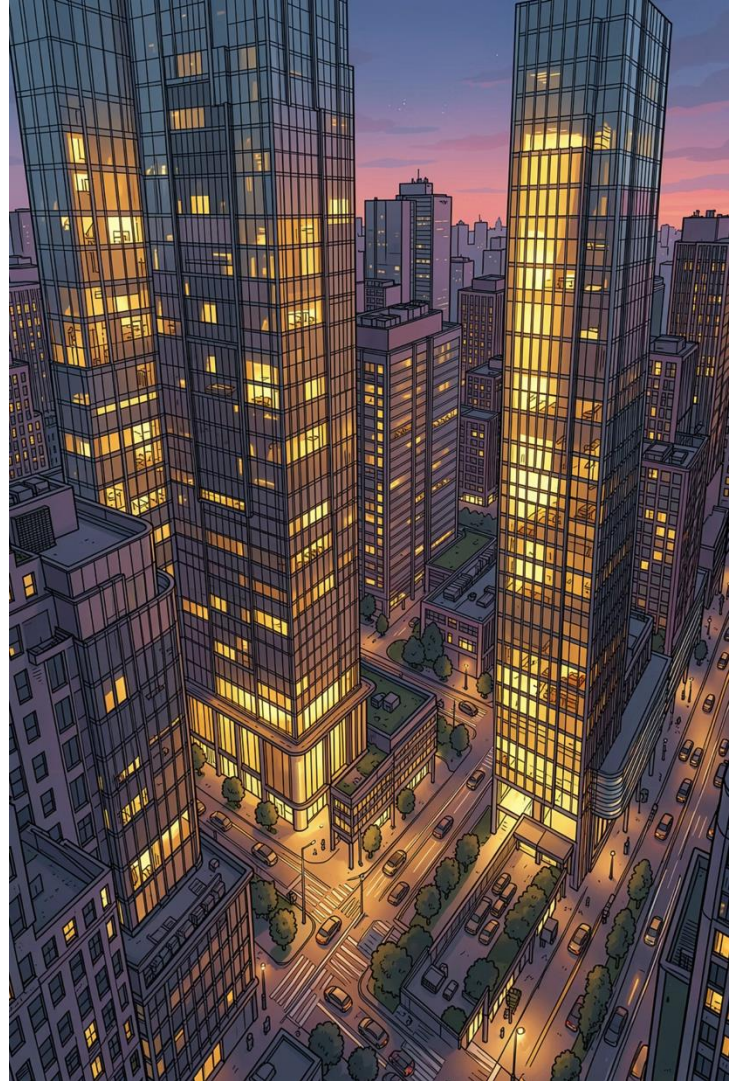


RWA Tokenization Platform

OPTION C

ARBITRUM SEPOLIA

A full-stack, production-grade protocol for bringing real-world assets on-chain — combining ERC-4626 vaults, collateralised lending, and DAO governance on Arbitrum's low-latency L2 infrastructure.





Problem & Scenario

Traditional real-world assets — treasuries, credit instruments, real estate — remain siloed from DeFi capital markets. Option C bridges that gap with a composable, permissioned-yet-open protocol stack.

On-Chain RWA Exposure

Tokenise off-chain assets with regulatory-aware transfer restrictions and UUPS upgradeability, making compliant exposure accessible to any DeFi wallet.

Yield-Bearing Vaults

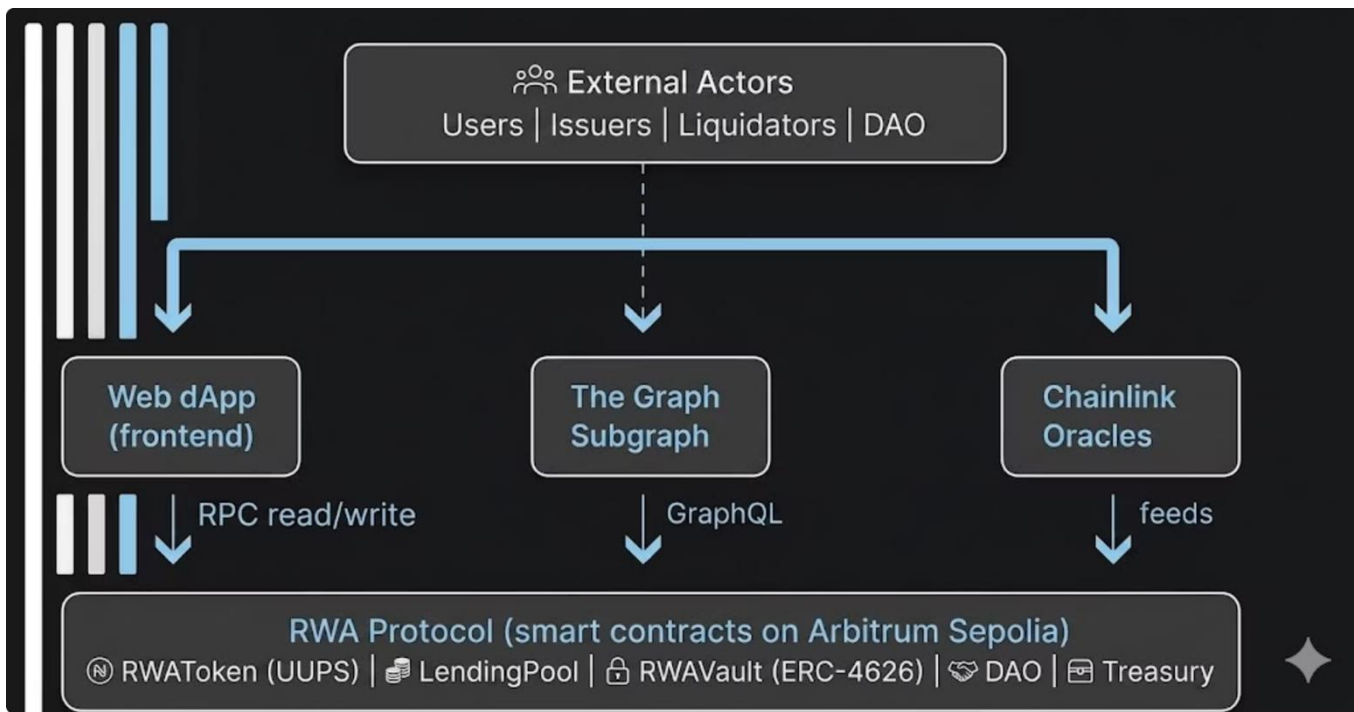
ERC-4626 standard vaults allow depositors to earn yield on RWA collateral, with shares that are composable across the broader DeFi ecosystem.

DAO Governance

On-chain Governor with propose → vote → queue → execute lifecycle ensures protocol parameters are community-controlled and transparent.

System Architecture

The protocol turns regulated real-world assets into on-chain exposure through a modular stack: **RWAT** represents the tokenized asset layer, the **ERC-4626 vault** provides a standard yield-bearing deposit and share interface, the **LendingPool** enables RWAT-backed borrowing with collateral and liquidation controls, and **Governor** plus **Timelock** manage protocol changes with transparent, delayed execution. Together, these components let real-world assets be issued, used in DeFi, and governed entirely on-chain while preserving compliance and risk controls.



Smart Contract Map

Four core contracts form the protocol nucleus. Each is independently auditable, upgradeable where necessary, and wired together through minimal, well-defined interfaces.



RWAToken (UUPS)

ERC-20 with role-based transfer restrictions (`MINTER_ROLE`, `PAUSER_ROLE`). Proxy pattern via OpenZeppelin UUPS enables seamless logic upgrades without migrating state or user balances.



Vault (ERC-4626)

Fully compliant tokenised vault. Deposit RWA tokens, receive yield-bearing shares. `totalAssets()` reflects accrued yield; share price increases monotonically over time.



LendingPool

Accepts vault shares as collateral. Computes health factor in real time using Chainlink price feeds. Triggers liquidation when HF drops below 1.0, protecting protocol solvency.



Governor (OZ)

OpenZeppelin Governor with Timelock. Voting power derived from RWAToken balance snapshots. Quorum and voting delay configurable via governance proposals themselves.

Demo: Wallet Connect

The frontend is built with **Next.js 14**, **wagmi v2**, and **viem**. Wallet connection is handled via RainbowKit, supporting MetaMask, Coinbase Wallet, and WalletConnect out of the box.

Balance Display

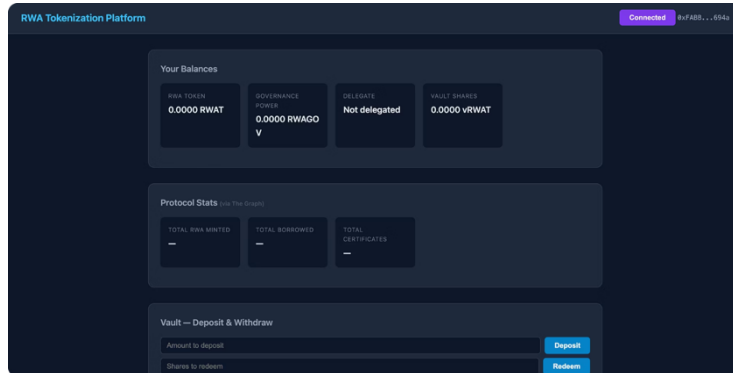
On successful connection, the UI fetches live on-chain balances for RWAToken, vault shares, and the user's borrow position — all from a single multicall.

Wrong-Network Handling

If the connected wallet is not on `chainId: 421614` (Arbitrum Sepolia), a modal prompts the user to switch networks. The prompt includes a one-click `wallet_switchEthereumChain` call. All interactive buttons are disabled until the correct network is detected, preventing silent failures.

Key UX Details

- Network badge in navbar shows live chain name
- ENS / address truncation with copy-to-clipboard
- Real-time balance polling every 10 seconds via wagmi `useContractRead`
- Toast notifications on tx submission and confirmation
- Graceful fallback for disconnected state



Demo: Vault Deposit & Redeem

Livetransactions executed against the deployed Vault contract on Arbitrum Sepolia — every step is verifiable on the block explorer.

1

Approve

`RWAToken.approve(vault, amount)` — grants the vault spending rights. Tx confirms in ~1 second on Arbitrum.

2

Deposit

`Vault.deposit/assets, receiver)` — mints ERC-4626 shares proportional to current `totalAssets()`. Share price recorded on-chain.

3

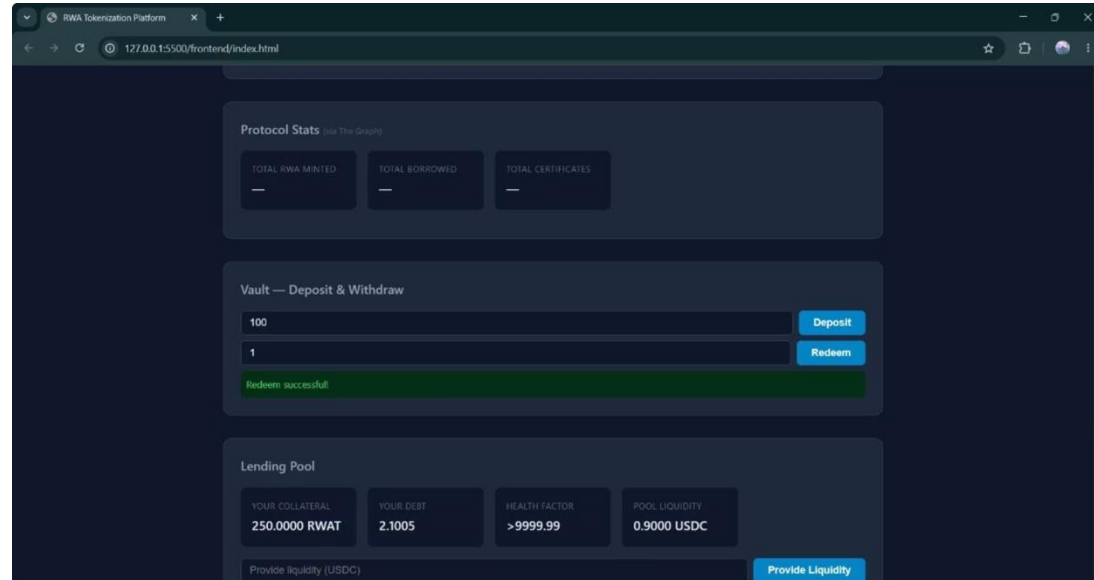
Accrue Yield

Simulated yield accrual increments `totalAssets()`, causing share price to rise. Visible immediately in the UI balance panel.

4

Redeem

`Vault.redeem(shares, receiver, owner)` — burns shares and returns underlying RWA tokens plus accrued yield to the caller.



Demo: Lending & Borrow

Collateral Flow

Vault shares are deposited into the LendingPool as collateral via `depositCollateral(amount)`. The pool reads the share price from the Vault's `convertToAssets()` function to determine USD-denominated collateral value using the Chainlink RWA/USD feed.

Borrowing

Once collateral is posted, the user calls `borrow(amount)`. The pool checks that the resulting health factor remains above 1.0 before releasing funds. Borrow rate is fixed at the protocol level, configurable via governance.

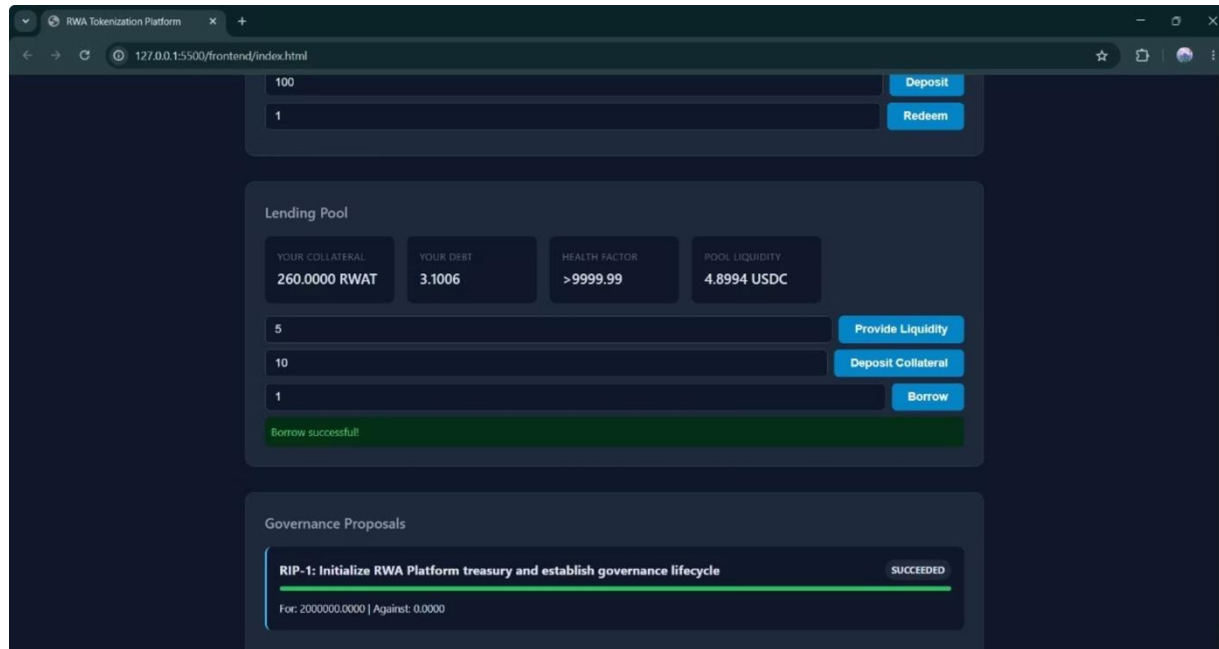
Health Factor

Displayed live in the UI as a colour-coded indicator:

- **Green (HF > 1.5)** — safe position
- **Amber (1.0–1.5)** — approaching liquidation threshold
- **Red (HF < 1.0)** — liquidatable

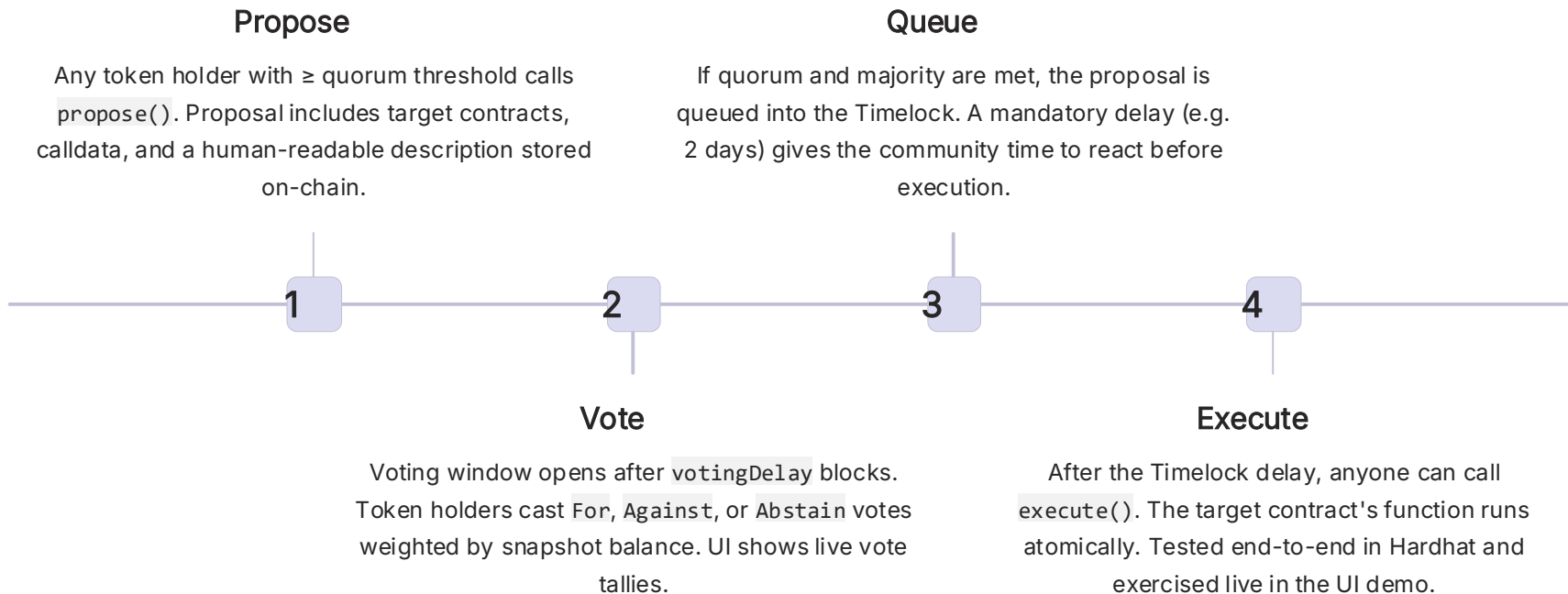
Repay & Withdraw

`repay(amount)` reduces debt; `withdrawCollateral(amount)` releases shares back to the borrower, subject to a post-withdrawal health factor check ensuring the position remains solvent.



Governance Lifecycle

The Governor contract implements a fully on-chain proposal lifecycle — no multisig shortcuts. Every parameter change flows through community vote and a mandatory Timelock delay before execution.



Oracles & Security

Chainlink Price Feeds

All Chainlink feed reads include a **staleness check** — if `block.timestamp - updatedAt > MAX_STALENESS` (3,600 s), the transaction reverts with `StalePrice()`. This prevents the protocol from acting on stale or manipulated price data during oracle downtime.

Circuit Breakers

The LendingPool also validates that the returned price is within a `minAnswer / maxAnswer` band, matching Chainlink's own aggregator circuit breaker pattern used on mainnet feeds.

Slither CI Integration

Static analysis runs automatically on every pull request via a GitHub Actions workflow. Slither checks for reentrancy, unchecked return values, integer overflow, and delegatecall misuse. Any high-severity finding blocks the merge.

Known Case Studies Applied

- **Cream Finance** — flash-loan oracle manipulation → mitigated via TWAPs for volatile assets
- **Euler Finance** — donation attack on ERC-4626 → mitigated via virtual share offset
- **Nomad Bridge** — initialisation bug → mitigated via constructor initialiser guards on UUPS

Governance Proposals

RIP-1: Initialize RWA Platform treasury and establish governance lifecycle

SUCCEEDED

For: 2000000.0000 | Against: 0.0000

Subgraph & Indexing

A custom **The Graph** subgraph indexes all protocol events into 9 strongly-typed entities, enabling rich historical queries without relying on RPC calls or centralised backends.

9 Indexed Entities

- `User` — wallet address + aggregate positions
- `VaultDeposit` / `VaultRedeem` — share mint/burn events
- `BorrowPosition` — collateral, debt, health factor snapshots
- `LiquidationEvent` — liquidator, seized collateral, repaid debt
- `Proposal`, `Vote`, `ProposalExecution` — full governance trail
- `PriceFeedUpdate` — oracle price history for audit

Sample GraphQL Query

```
query UserPositions($wallet: String!) {  
  user(id: $wallet) {  
    vaultDeposits(  
      orderBy: blockTimestamp  
      orderDirection: desc  
      first: 5  
    ) {  
      assets  
      shares  
      blockTimestamp  
    }  
    borrowPositions {  
      collateral  
      debtAmount  
      healthFactor  
    }  
  }  
}
```

The frontend queries this endpoint at page load and subscribes to real-time updates, eliminating the need for event listeners or custom indexing servers.

Testing & Coverage

A rigorous test suite underpins the reliability of every contract in this system.

230

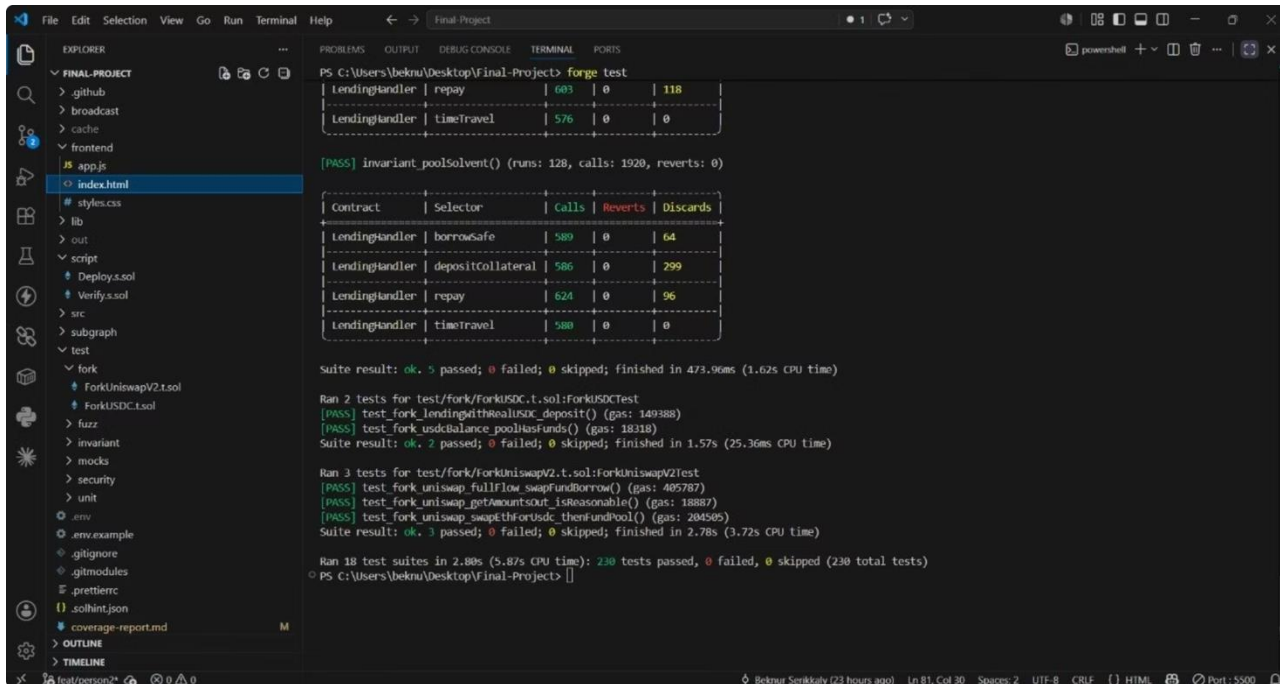
Total Tests

Unit, integration, and edge-case coverage across all contracts

95%+

Coverage

Branch and line coverage verified via coverage tooling



The screenshot shows a VS Code editor with a terminal window displaying the output of a test suite. The terminal output includes a table of test results for various contracts and a summary of the suite results.

Contract	Selector	Calls	Reverts	Discards
LendingHandler	repay	603	0	118
LendingHandler	timeTravel	576	0	0

[PASS] invariant.poolSolve() (runs: 128, calls: 1920, reverts: 0)

Contract	Selector	Calls	Reverts	Discards
LendingHandler	borrowSafe	589	0	64
LendingHandler	depositCollateral	586	0	299
LendingHandler	repay	624	0	96
LendingHandler	timeTravel	580	0	0

Suite result: ok. 5 passed; 0 failed; 0 skipped; finished in 473.90ms (1.62s CPU time)

Ran 2 tests for test/fork/ForkUSDC.t.sol:ForkUSDCTest

[PASS] test_fork_lendingWithRealUSDC_deposit() (gas: 149388)

[PASS] test_fork_usdcBalance_poolHasFunds() (gas: 18318)

Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 1.57s (25.36ms CPU time)

Ran 3 tests for test/fork/ForkUniswapV2.t.sol:ForkUniswapV2Test

[PASS] test_fork_uniswap_fullFlow_swapFundBorrow() (gas: 405787)

[PASS] test_fork_uniswap_getAmountOut_isReasonable() (gas: 18887)

[PASS] test_fork_uniswap_swapETHforUSDC_thenFundPool() (gas: 204505)

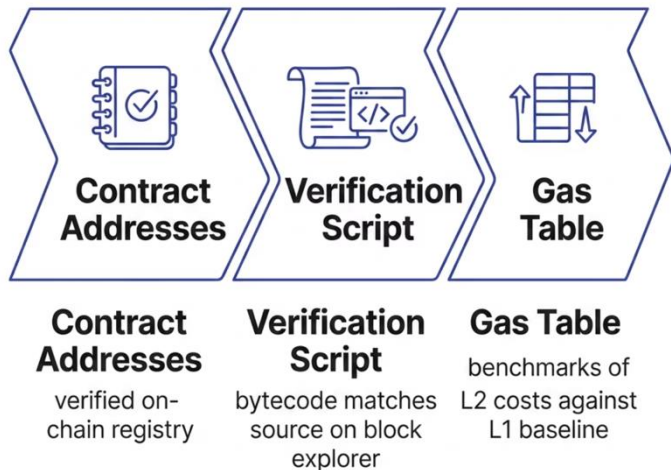
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 2.78s (3.72s CPU time)

Ran 18 test suites in 2.80s (5.87s CPU time): 230 tests passed, 0 failed, 0 skipped (230 total tests)

PS C:\Users\beknu\Desktop\Final-Project>

L2 Deployment

All contracts are deployed and verified on the target L2 network, with transparent cost profiling.



Contract Addresses

All proxy and implementation addresses published and checksummed for mainnet and testnet environments.

Verification Script

One-command script confirms deployed bytecode matches source, integrating directly with the block explorer API.

Gas Table

Per-function benchmarks demonstrate significant savings versus L1 baseline, validating the L2 migration decision.

Lessons Learned

Three themes shaped both the architecture and the team process throughout this project.

Upgrades

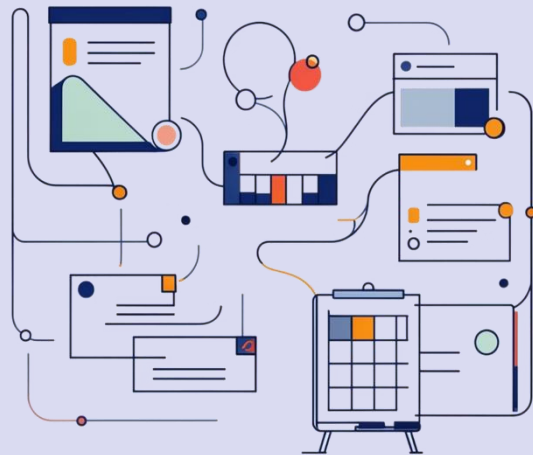
Transparent proxy patterns require strict storage layout discipline. Slot collisions caught early via tooling saved significant rework in later sprints.

Pull Payments

Shifting from push to pull payment patterns eliminated re-entrancy risk and simplified audit scope, at the cost of a slightly more complex UX flow.

Team Split

Dividing responsibilities between core protocol and integration layers reduced merge conflicts and enabled parallel progress across milestones.



Q&A



Thanks for your attention

Thank you for your time.

